

Understanding graph representation learning in RL-based logic synthesis



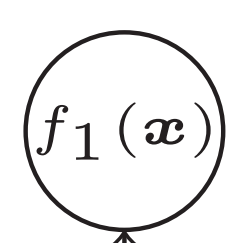
Hector Kohler, Johannes Lutzeyer, Jesse Read

LIX, École Polytechnique, CNRS, Institut Polytechnique de Paris

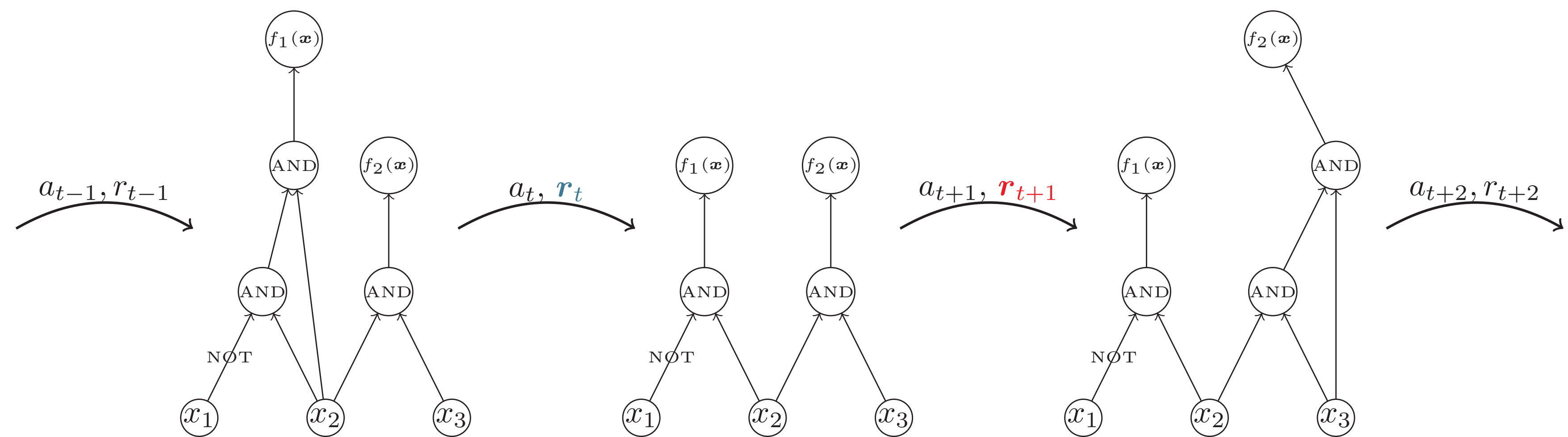
1. Logic synthesis with AIGs

Smaller circuits are cheaper to manufacture (Ou et al., 2024). Logic synthesis represents a circuit as an **and-inverter graph** (AIG): a DAG whose internal nodes are *AND* gates and whose edges may carry a *NOT*. Every Boolean function has an AIG.

Finding the *smallest* AIG lies in NP but is not known to be NP-hard (Kabanets and Cai, 2000): no exact solver exists at the sizes of interest, so there is no optimal AIG to imitate. ABC (Brayton and Mishchenko, 2010) instead applies sequences of local rewrites (Mishchenko et al., 2006) that preserve functional equivalence and remove *AND* nodes.



2. AIG minimization as a Markov decision process

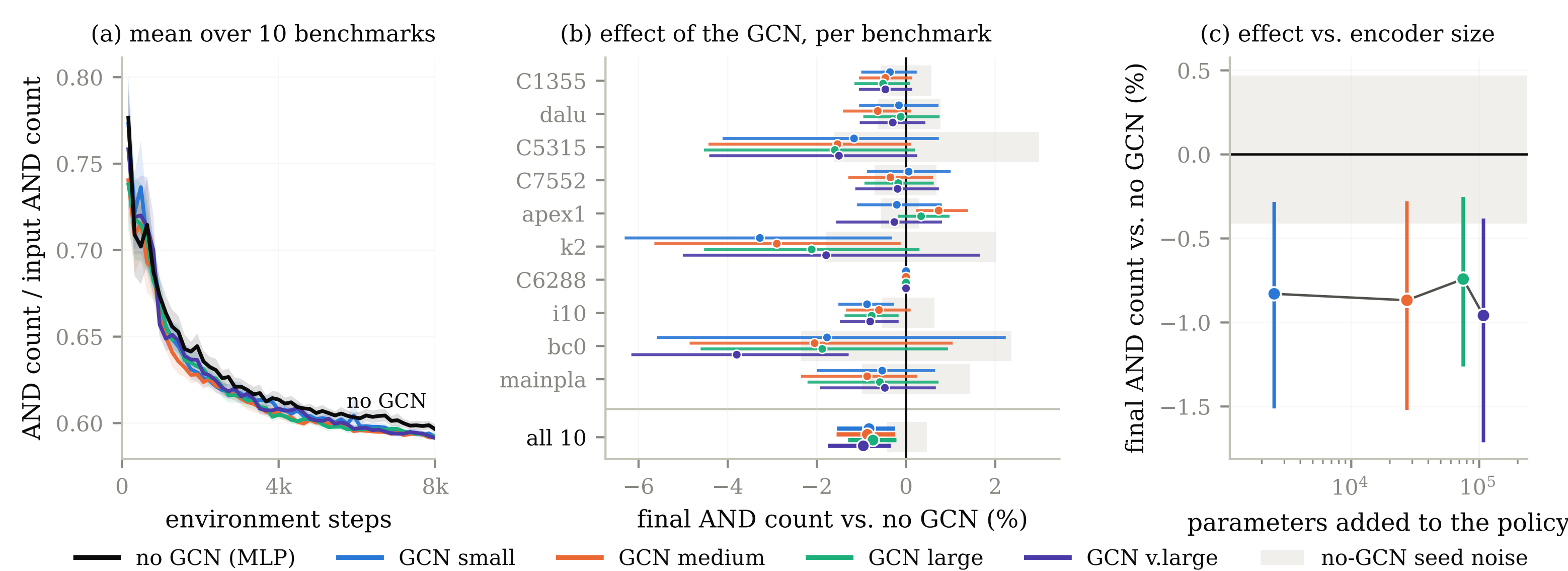


A trajectory on the AIGs of f . A state is an AIG, an action is one ABC primitive applied to the whole graph, and the reward is the drop in *AND* count: the agent is paid for shrinking the circuit (green) and penalized for growing it (red). Primitives preserve functional equivalence, so every reachable state still computes f .

We reuse the MDP of Zhu et al. (2020): the actions are the five ABC primitives **balance**, **rewrite**, **rewrite -z**, **refactor** and **refactor -z**, transitions are deterministic, the reward is $R(s_t, a_t) = 1 - n_{t+1}/n_0$ with n_t the *AND* count, and an episode applies exactly $H = 20$ primitives — a search over flows of length 20 over a five-letter alphabet. Reinforcement learning and planning for logic synthesis (Zhu et al., 2020; Hosny et al., 2020; Neto et al., 2023) all act at this granularity; generative models that output a reduced AIG directly do not yet work at these sizes (Stoll et al., 2026).

3. Does a graph encoder help PPO?

We keep the pipeline of Zhu et al. (2020) fixed — MDP, policy and value heads, PPO (Schulman et al., 2017), hyper-parameter grid — and vary **only how the AIG is shown to the policy**: the graph statistics x_G (*AND* and level counts of the current and previous AIG, recent actions, normalized time step) read by an MLP, against four GCN encoders adding 2,496 to 107,968 parameters on top of them. Ten benchmarks of 504 to 6,359 *AND* gates, every (AIG, encoder) pair tuned over the same grid and reported at its best cell.



(a) *AND* count averaged over the ten benchmarks; (b) change in final *AND* count against the no-GCN policy, per benchmark and pooled (*all 10*), the grey band being the no-GCN policy's own spread over its five seeds; (c) the pooled change against the parameters the encoder adds. Intervals are 95% bootstrap confidence intervals over five seeds.

Tuned PPO minimizes AIGs **regardless of the architecture**, a median 6% below the **resyn2** heuristic (Panda and Najm, 1998). Pooled, the GCN policies do end 0.74%–0.96% below the MLP with intervals excluding zero, but the effect is marginal: no larger than re-running the baseline with other seeds (grey band), only six of the forty encoder-benchmark comparisons have an interval entirely below zero, and the best GCN saves a median of 11 *AND* gates. Crucially, the gain **does not grow with encoder size**: 2,496 added parameters help as much as 107,968. On the 100 circuits (106 to 46,977 *AND* gates) of the 2026 IWLS contest the picture is the same, 0.3%–0.7% below the MLP.

5. Why does it help so little?

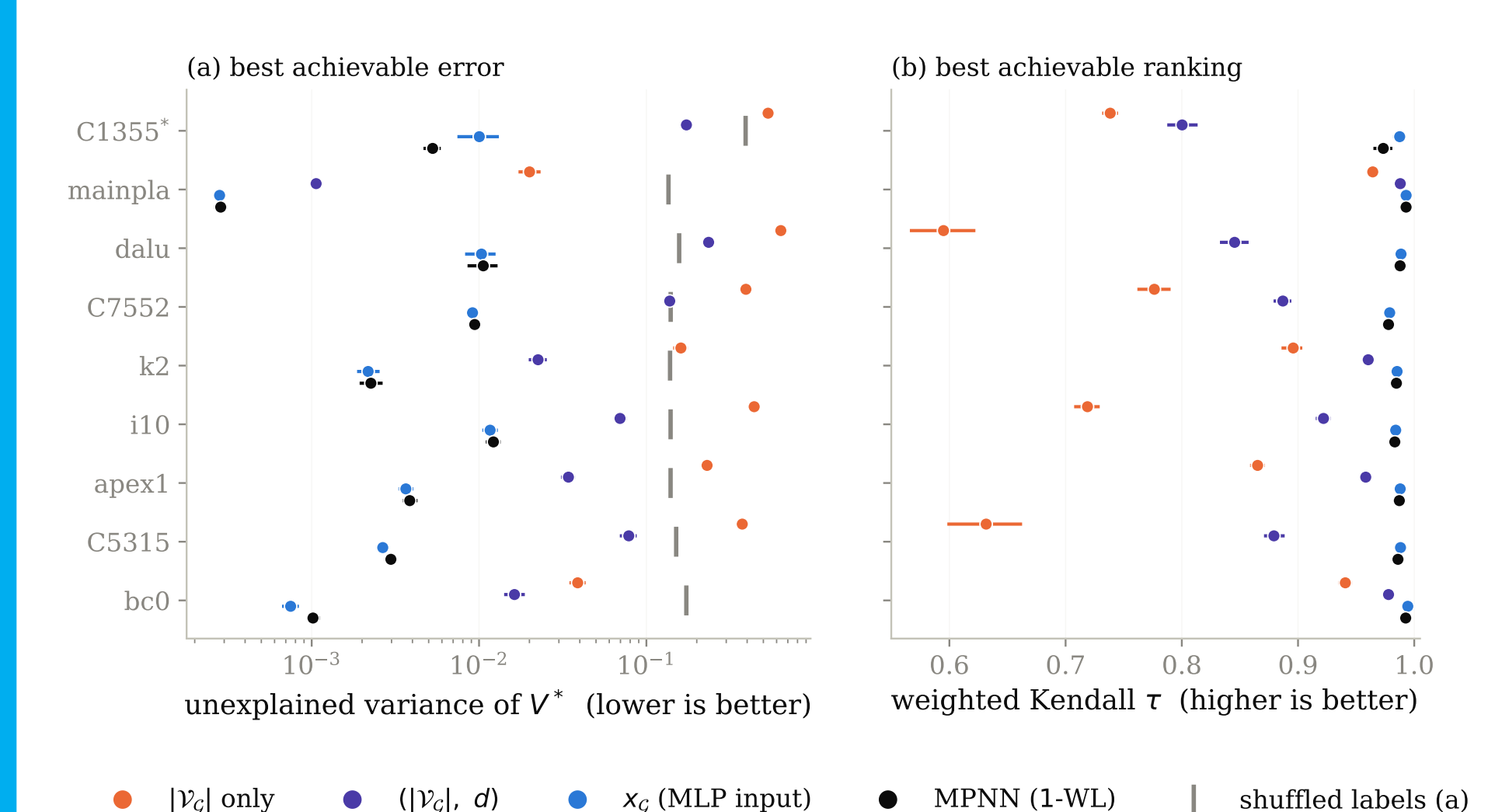
A null result inside one algorithm could be a property of that algorithm. We therefore ask a question with no training algorithm in it: **how well can the**

4. Final AIG sizes

AIG	Starting nodes	no GCN (MLP)	GCN small	GCN medium	GCN large	GCN v. large	resyn2
C1355	504	389 ± 3	387 ± 1	387 ± 0	387 ± 1	387 ± 0	390
dalu	1740	1008 ± 9	1006 ± 6	1001 ± 3	1006 ± 6	1005 ± 2	1052
C5315	1773	1303 ± 43	1287 ± 8	1283 ± 2	1282 ± 6	1283 ± 5	1309
C7552	2074	1371 ± 12	1372 ± 10	1367 ± 11	1369 ± 5	1369 ± 10	1455
apex1	2124	1066 ± 7	1064 ± 1	1074 ± 4	1070 ± 4	1063 ± 14	1179
k2	2190	1049 ± 26	1015 ± 32	1019 ± 27	1027 ± 31	1030 ± 36	1194
C6288	2337	1870 ± 0	1870 ± 0	1870 ± 0	1870 ± 0	1870 ± 0	1870
i10	2675	1715 ± 13	1700 ± 3	1705 ± 9	1702 ± 3	1702 ± 6	1829
bc0	3501	1512 ± 46	1485 ± 59	1481 ± 31	1484 ± 28	1455 ± 13	1713
mainpla	6359	2318 ± 37	2306 ± 14	2298 ± 11	2305 ± 25	2307 ± 14	2476

Mean *AND* count after $H = 20$ ABC primitives, over the five seeds of each configuration's best hyper-parameter cell, $\pm$ one standard deviation across those seeds. *Starting nodes* is the count before any primitive is applied; **resyn2** is deterministic and has no spread. AIGs are ordered by input size.

6. Expressivity bounds



Best error (a) and best ranking (b) achievable by each model class on $\hat{V}$, over nine benchmarks and ten dataset seeds. **C6288** is excluded, its target taking only two distinct values; ***C1355** has only six.

Message passing is worth a great deal *over global descriptors*: the bound falls from 0.02–0.64 of the target variance for node count alone to 3×10^{-4} – 1.2×10^{-2} for 1-WL. It is worth **nothing over x_G** . On eight of the nine benchmarks the x_G bound is the *lower*